Portfolio 3: Transactions and Concurrency

Project: Stretto Technologies Database System

Database Environment: PostgreSQL (pgAdmin 4)

1. Transaction Management: COMMIT and ROLLBACK

In the Stretto Technologies database, processing a customer order requires inserting data into multiple related tables, specifically invoices and invoice_items. To maintain data integrity (the ACID properties), these inserts must be handled as a single logical transaction. If one part fails, the entire transaction must abort to prevent orphan records.

1.1 Successful Transaction (COMMIT): This transaction simulates a successful customer purchase. An invoice is created, and two line items are attached. Because no errors occur and all NOT NULL constraints (such as customer_id, create_date, and total_amount) are met, the transaction is committed permanently to the database.

SQL

```
BEGIN;

-- 1. Create the main invoice record

INSERT INTO invoices (invoice_number, customer_id, create_date, subtotal, shipping_fee, total_amount)

VALUES ('INV_TXN_001', 1, CURRENT_TIMESTAMP, 1500.00, 50.00, 1550.00);

-- 2. Add the first sourced device

INSERT INTO invoice_items (invoice_number, item_id, supplier_id, quantity, unit_price, cost_price, device_identifiers)

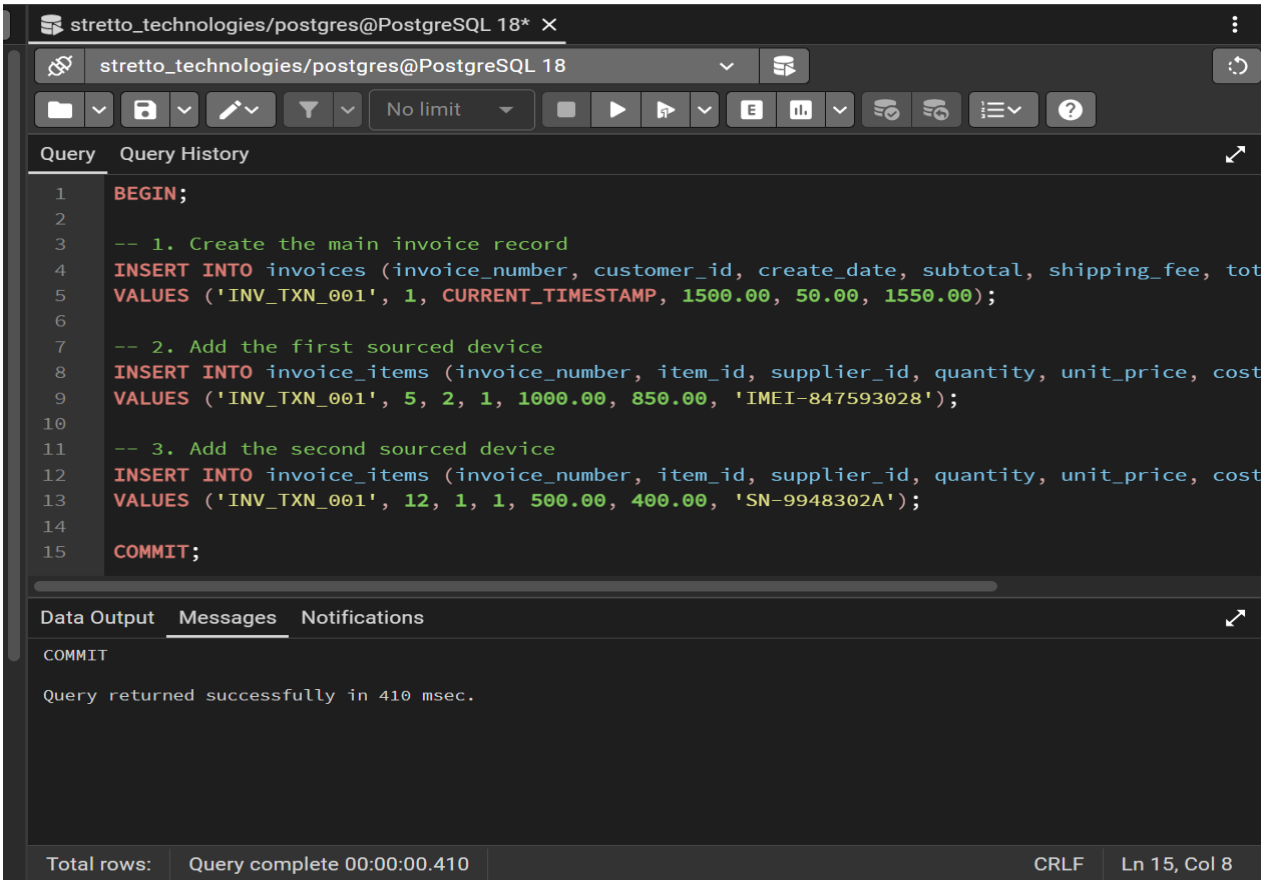
VALUES ('INV_TXN_001', 5, 2, 1, 1000.00, 850.00, 'IMEI-847593028');

-- 3. Add the second sourced device

INSERT INTO invoice_items (invoice_number, item_id, supplier_id, quantity, unit_price, cost_price, device_identifiers)

VALUES ('INV_TXN_001', 12, 1, 1, 500.00, 400.00, 'SN-9948302A');
```

COMMIT;



1.2 Failed Transaction (ROLLBACK): This scenario simulates an application error where an invoice is created, but the line item insertion violates a constraint. The invoice_items table strictly enforces that unit_price is NOT NULL. Attempting to insert a null value causes PostgreSQL to catch the error, and the ROLLBACK command is issued to undo the initial invoice creation, ensuring no partial data is saved.

SQL

BEGIN;

-- 1. Create the main invoice record

INSERT INTO invoices (invoice_number, customer_id, create_date, subtotal, shipping_fee, total_amount)

VALUES ('INV_TXN_002', 2, CURRENT_TIMESTAMP, 800.00, 20.00, 820.00);

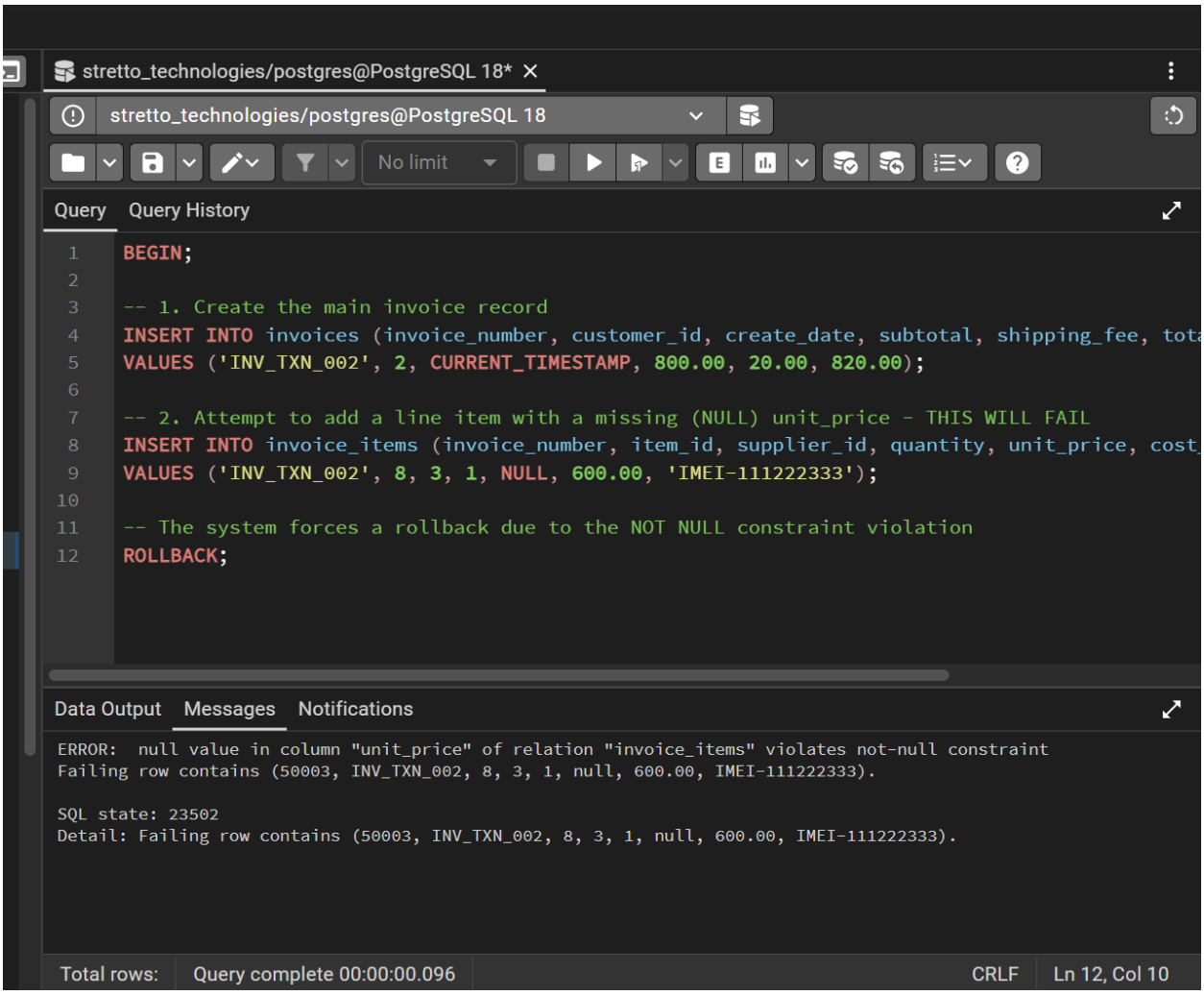
-- 2. Attempt to add a line item with a missing (NULL) unit_price - THIS WILL FAIL

INSERT INTO invoice_items (invoice_number, item_id, supplier_id, quantity, unit_price, cost_price, device_identifiers)

VALUES ('INV_TXN_002', 8, 3, 1, NULL, 600.00, 'IMEI-111222333');

-- The system forces a rollback due to the NOT NULL constraint violation

ROLLBACK;



2. Concurrency Control Mechanism

In a dynamic e-commerce environment, concurrency issues like "Lost Updates" occur if two procurement officers attempt to log a payment for the same invoice simultaneously. To prevent this, Stretto Technologies utilizes **Row-Level Locking** (SELECT ... FOR UPDATE).

Scenario: An administrator is processing a payment for an invoice into the payments table. By applying FOR UPDATE, the database locks that specific invoices row. If a second user attempts

to modify the same invoice, their transaction is forced to wait until the first user completes their transaction, preventing overlapping financial anomalies.

SQL

```
BEGIN;

-- Lock the specific invoice row to prevent other sessions from modifying it concurrently

SELECT invoice_number, total_amount

FROM invoices

WHERE invoice_number = 'INV_TXN_001'

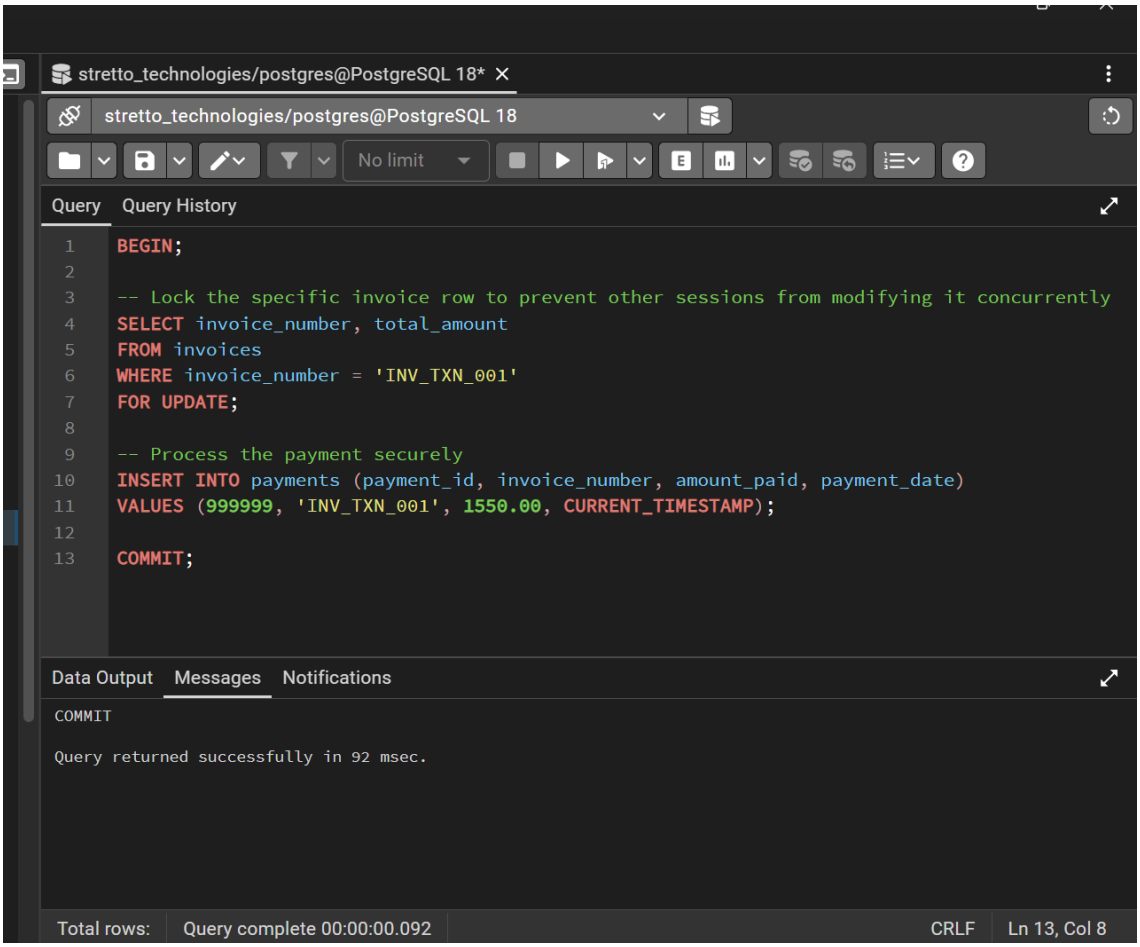
FOR UPDATE;

-- Process the payment securely

INSERT INTO payments (payment_id, invoice_number, amount_paid, payment_date)

VALUES (999999, 'INV_TXN_001', 1550.00, CURRENT_TIMESTAMP);

COMMIT;
```



3. Transaction Isolation Levels

PostgreSQL defaults to the READ COMMITTED isolation level, which is generally sufficient. However, for highly sensitive financial operations—such as generating Stretto Technologies' end-of-month profit reports across all transactions—the system requires the strictest isolation level to prevent "Phantom Reads" (where new rows are added by other users in the middle of a reporting query).

Scenario: We elevate the isolation level to SERIALIZABLE. This ensures the transaction behaves as if it is the only one running on the database, guaranteeing absolute mathematical accuracy for financial auditing of the total_amount field.

SQL

-- Set strict isolation for financial reporting

BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Calculate total revenue for a specific period

SELECT SUM(total_amount) AS total_monthly_revenue

FROM invoices

WHERE create_date >= '2026-03-01';

COMMIT;

